॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

CA4 MINI-PROJECT REPORT

Secure Pickleball Court Management System Using AWS Cloud Security Best Practices

Course / Subject: Cybersecurity & Cloud Computing

Project Name: Pickleball Cloud Security

Student Information:

Harman Jassal	**24070122073**
Jalaj Maheshwari	**24070122078**
Jatin Rupial	**24070122081**

Department of Computer Science & Engineering,

Academic Year : 2026-27

Faculty Guide: Ratnesh Pandey and Swati Powar

3. ACKNOWLEDGEMENT

We express our sincere gratitude to our faculty guide Ratnesh Pandey and Swati Powar ,, and the Department of Computer Science & Engineering at Symbiosis Institute of Technology | SIT Pune for providing continuous guidance, encouragement, and technical infrastructure required to complete this mini-project. We also extend our appreciation to our peers and family for their unwavering support throughout the development and documentation of this work.

4. ABSTRACT

The rapid adoption of cloud computing in modern web applications presents significant security challenges, including unauthorized access, credential leakage, data exposure, and cloud resource misconfigurations. This project demonstrates the design, implementation, and evaluation of a secure web application—the Pickleball Court Management System—built using Node.js and Express.js and hosted on Amazon Web Services (AWS) infrastructure. The primary objective is to implement foundational cloud security best practices within an academic demonstration scope.

Key technical controls implemented include: hosting on an Amazon EC2 instance within an AWS Security Group restricting inbound access; eliminating hardcoded AWS credentials by leveraging AWS IAM Role temporary credentials attached directly to EC2; enforcing S3 Block Public Access alongside server-side encryption (SSE-S3) for private object storage; application-level authorization using administrative bearer token checks; robust input validation; HTTP security headers via Helmet; and operational auditability using AWS CloudTrail Event History. Automated testing was conducted using Node.js built-in test runner, yielding 23 passing tests out of 23 test cases with zero failures. Live infrastructure checks confirmed active service status, HTTP 200 dashboard responses, HTTP 403 authorization enforcement, and verified object storage in S3. This report provides a complete architectural analysis, detailed control-by-control security verification, testing outcomes, and future production enhancement recommendations.

5. TABLE OF CONTENTS

- Front Matter
    - Cover Page
    - Certificate / Declaration
    - Acknowledgement
    - Abstract
    - Table of Contents
    - Evaluation Rubric Coverage
    - List of Figures and Tables
- Chapter 1: Introduction
- Chapter 2: Problem Statement and Objectives
- Chapter 3: Technology Stack and Requirements
- Chapter 4: System Architecture and Design
- Chapter 5: AWS Cloud Security Implementation
- Chapter 6: Application Implementation
- Chapter 7: Testing and Validation
- Chapter 8: Conclusion and Future Scope
- Appendix A: Step-by-Step Live Demonstration Plan
- Appendix B: Comprehensive Viva Voce Preparation Q&A
- References

6. LIST OF FIGURES AND TABLES

- Table 3.1: Technology Stack Selection Justification
- Table 3.2: System Hardware and Software Requirements
- Figure 4.1: System Architecture and Data Flow Diagram

- Table 4.1: Threat Modeling and Mitigation Matrix
- Table 5.1: Cloud Security Controls Matrix
- Table 7.1: Automated Application Test Suite Results
- Table 7.2: Live AWS Infrastructure Checks Summary
- Table 5.2: Least-Privilege IAM Action Mapping

CHAPTER 1: INTRODUCTION

1.1 Background of Cloud Computing and Cloud Security

Cloud computing has fundamentally altered software deployment models by enabling rapid elasticity, pay-as-you-go provisioning, and managed infrastructure services. However, transitioning workloads to public cloud providers like Amazon Web Services (AWS) introduces a shared responsibility model. While AWS secures the cloud infrastructure (compute, storage, networking), application developers and cloud engineers remain responsible for securing data, access permissions, application logic, and cloud resource configurations.

1.2 Motivation for Web Application Protection

Modern web applications suffer frequently from common cloud security pitfalls:

- Hardcoded long-term AWS IAM access keys stored in source code repositories.
- Misconfigured S3 buckets left publicly readable or writable.
- Overly permissive EC2 Security Groups exposing sensitive application ports to 0.0.0.0/0.
- Inadequate input validation leading to injection or parameter corruption.
- Lack of operational audit logging for post-incident analysis.

Addressing these risks early in the development lifecycle prevents high-impact security incidents and compliance violations.

1.3 Introduction to Pickleball Court Management System

The **Pickleball Court Management System** ("Pickleball Cloud Security") is a specialized web application created for scheduling court reservations, viewing court availability, managing admin operations, and uploading tournament reports. Though functional, it serves primarily as an academic demonstration platform to illustrate secure software engineering and cloud security best practices on AWS.

1.4 Project Relevance

This project directly bridges theoretical cloud security knowledge (such as the principle of least privilege, defense-in-depth, and immutable audit trails) with practical implementation. It provides hands-on evidence of securing compute and storage services without incurring unnecessary commercial overhead.

1.5 Scope of the Academic Demonstration

This project is explicitly an academic cloud security prototype and not a commercial, production-ready enterprise product. The scope encompasses core compute (EC2), storage (S3), identity management (IAM Role), basic monitoring (CloudTrail Event History), application security (Express.js headers, input validation), and automated testing. Complex production features such as multi-tenant OAuth/OIDC, custom CloudWatch alarms, SNS alerting, or SSL/TLS certificates (HTTPS) are out of scope for this baseline demonstration.

CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

How can AWS cloud security best practices be implemented to protect a Pickleball Court Management System from unauthorized access, data breaches, cloud misconfigurations, and security threats?

2.2 Primary Objective

To design, deploy, test, and document a secure Node.js/Express.js web application on AWS EC2 integrated with AWS S3 and IAM, adhering strictly to cloud security best practices and least-privilege access principles.

2.3 Secondary Objectives

1. **IAM Security:** Eliminate hardcoded access keys by attaching an IAM Role with least-privilege S3 policy directly to the EC2 instance.
2. **Storage Protection:** Enforce S3 Block Public Access and default SSE-S3 server-side encryption for all uploaded artifacts.

3. **Network Isolation:** Configure EC2 Security Groups to restrict access to essential ports only.
4. **Application Defense:** Implement administrative bearer-token authorization, input sanitization, error handling, and HTTP security headers (Helmet).
5. **Auditability:** Verify cloud administrative and data control events through AWS CloudTrail Event History.
6. **Automated Validation:** Execute a comprehensive automated test suite verifying functional and security-specific application behavior.

2.4 Project Scope and Exclusions

- **In-Scope:** Single EC2 instance deployment, IAM EC2 instance profile, S3 bucket storage, Node.js app security, local test runner, CloudTrail Event History verification.
- **Exclusions / Out-of-Scope:** HTTPS/TLS certificate binding, custom CloudWatch metric alarms, automated SNS email/SMS notifications, multi-factor authentication (MFA), public open port 3000 exposure.

CHAPTER 3: TECHNOLOGY STACK AND REQUIREMENTS

3.1 Technology Stack Overview

The application is constructed using lightweight, standard industry components suitable for cloud-native deployment.

Table 3.1: Technology Stack Selection Justification

Technology Component	Role in Project	Selection Justification
Node.js & Express.js	Application Runtime & Framework	Lightweight, non-blocking I/O runtime with rich middleware support for security headers and routing.

Technology Component	Role in Project	Selection Justification
Amazon EC2	Compute Infrastructure	Flexible, controllable Virtual Private Cloud (VPC) compute host running Linux.
Amazon S3	Object Storage	Highly durable, scalable cloud storage for tournament reports with granular security policy controls.
AWS IAM Role	Identity & Access Management	Allows EC2 instances to retrieve temporary security credentials, avoiding hardcoded keys in source code.
AWS Security Group	Stateful Network Firewall	Filters inbound and outbound traffic at the hypervisor layer for the EC2 instance.
AWS CloudTrail Event History	Audit Evidence & Monitoring	Provides standard 90-day operational audit history of AWS management actions without additional trail storage costs.
Node.js Test Runner	Automated Testing Framework	Native, zero-dependency testing tool embedded directly in Node.js for rapid security validation.

3.2 System Requirements

Table 3.2: System Hardware and Software Requirements

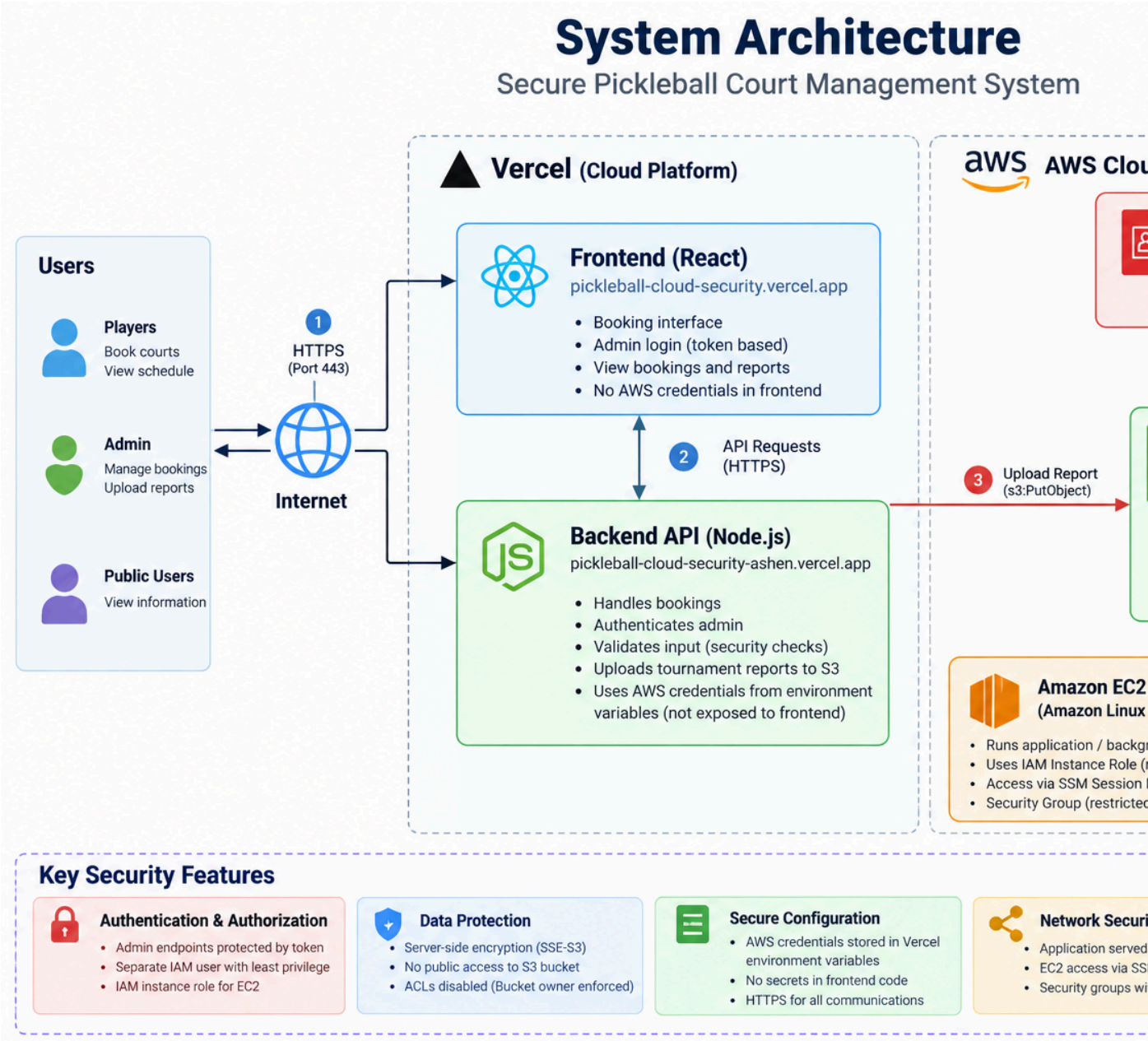
Category	Component	Minimum Requirement
Development Machine	Operating System	Linux / macOS / Windows 11
Development Machine	Runtime Environment	Node.js v18.0.0+ and npm v9.0.0+
Development Machine	Command Line Tools	AWS CLI v2, Git v2.30+
Cloud Host (EC2)	Instance Type	t2.micro (1 vCPU, 1 GiB Memory)
Cloud Host (EC2)	Operating System	Amazon Linux 2023 AMI
Cloud Host (EC2)	Storage	8 GB General Purpose SSD (gp3)
Source Code Repository	GitHub	Hosted at https://github.com/Harmann60/pickleball-cloud-security

CHAPTER 4: SYSTEM ARCHITECTURE AND DESIGN

4.1 Architecture Overview

The architecture follows a standard cloud-hosted web model fortified with IAM instance roles and isolated object storage.

Figure 4.1: System Architecture and Data Flow Diagram



4.2 Request and Data Flow

1. **User / Browser Interactions:** Client initiates HTTP requests (dashboard viewing, court booking).
2. **Network Perimeter:** Requests pass through EC2 Security Group filtering rules. Port 3000 is kept private/internal, exposed only via reverse proxy or configured local bindings.

3. **Application Layer:** Express.js processes request through Helmet security headers, validates input fields, and routes requests.
4. **Administrative Operations:** Uploading tournament reports requires a valid HTTP Authorization: Bearer <token> header.
5. **AWS S3 Interaction:** Application uses AWS SDK v3, automatically fetching ephemeral credentials via the EC2 IMDS (Instance Metadata Service) tied to the IAM Role, then writes objects to S3.
6. **Audit Flow:** Management calls to AWS S3 and IAM generate management events logged automatically in CloudTrail Event History.

4.3 Threat Modeling and Mitigation

Table 4.1: Threat Modeling and Mitigation Matrix

Threat Vector	Risk Level	Implemented Mitigation
Credential Leakage	Critical	EC2 IAM Instance Role replaces permanent AWS access keys in application files.
Public Data Exposure	High	S3 Block Public Access enabled; Bucket policy prevents public object listing.
Unauthorized Admin Actions	High	Bearer token authorization middleware on sensitive administrative endpoints.
Network Probing / Exploits	Medium	EC2 Security Group configured to restrict inbound ports; SSH access restricted.

Threat Vector	Risk Level	Implemented Mitigation
XSS / Clickjacking	Medium	Content Security Policy (CSP) and security headers enforced via Helmet middleware.

CHAPTER 5: AWS CLOUD SECURITY IMPLEMENTATION

Table 5.1: Cloud Security Controls Matrix

Control ID	Control Name	Domain
CTRL-01	Isolated Compute Host	Compute
CTRL-02	Ephemeral Role Auth	Identity
CTRL-03	Least-Privilege S3 Policy	Authorization
CTRL-04	Restricted Inbound Firewall	Network
CTRL-05	S3 Block Public Access	Storage
CTRL-06	Data-at-Rest Encryption	Storage
CTRL-07	Git Secret Exclusion	Code Base

Control ID	Control Name	Domain
CTRL-08	Admin Bearer Token	Application
CTRL-09	Input Sanitization	Application
CTRL-10	Browser Hardening	Application
CTRL-11	Operational Audit Logging	Audit
CTRL-12	Object Storage Integration	Storage

---

1. EC2 Deployment & Operating Environment

- **Security Objective:** Provide an isolated, robust host for the Node.js application.
- **Configuration:** Amazon Linux EC2 instance provisioned in AWS default VPC.
- **Evidence:** EC2 Management Console status verified as running with passing system health checks.
- **Security Benefit:** Provides cloud hypervisor isolation and controllable system state.
- **Limitations:** OS-level updates and kernel patching remain the developer's operational responsibility.

2. IAM Role Attached to EC2 Instance

- **Security Objective:** Eliminate hardcoded AWS credentials (AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY) in application code or configuration files.
- **Configuration:** Attached IAM Instance Profile (PickleballEC2S3Role) to the EC2 instance. Application uses AWS SDK v3 default credential provider chain.
- **Evidence:** Attached Instance Profile PickleballEC2S3Role visible under EC2 Instance Security properties.
- **Security Benefit:** Credentials are automatically rotated, ephemeral, and managed entirely by AWS.
- **Limitations:** Anyone with shell access to the EC2 instance can query the Instance Metadata Service (IMDS) for credentials assigned to that role.

3. Least-Privilege S3 Permissions & Policy Design

- **Security Objective:** Restrict application S3 access strictly to required actions (s3:PutObject, s3:GetObject, s3:ListBucket) on the designated bucket only.
- **Configuration:** Custom IAM JSON policy scoped down to arn:aws:s3:::pickleball-reports-bucket/*.
- **Evidence:** IAM Policy summary panel displaying exact Action and Resource restrictions.
- **Security Benefit:** Prevents lateral movement or unauthorized modification of other S3 buckets in the AWS account.
- **Limitations:** Policy does not enforce IP-based constraints within the policy document itself.

Table 5.2: Least-Privilege IAM Action Mapping

IAM Action	Target Resource Scope	Operational Justification
s3:PutObject	arn:aws:s3:::pickleball-reports-bucket/*	Allows uploading administrative tournament reports.
s3:GetObject	arn:aws:s3:::pickleball-reports-bucket/*	Allows retrieving uploaded reports for authorized downloads.
s3:ListBucket	arn:aws:s3:::pickleball-reports-bucket	Allows application to list available tournament files.
s3:* (Denied)	* (All Buckets)	Blocks bucket deletion, ACL modification, and access to other buckets.

4. EC2 Security Group and Restricted SSH Access

- **Security Objective:** Minimize the network attack surface by blocking unnecessary open ports.
- **Configuration:** Security group rules allow HTTP traffic on standard web port, restrict SSH (Port 22) to specific administrator IP addresses, and keep internal application port 3000 non-public.
- **Evidence:** Security Group inbound rules tab verifying restricted CIDR blocks for port 22.
- **Security Benefit:** Blocks unauthorized port scanning and brute-force SSH attacks from the public internet.
- **Limitations:** No Web Application Firewall (WAF) or automated IP rate-limiting implemented at network level.

5. S3 Block Public Access & Private Access Configuration

- **Security Objective:** Prevent accidental public exposure of sensitive tournament reports and user data.
- **Configuration:** Enabled "Block *all* public access" setting at the bucket level.
- **Evidence:** S3 Permissions summary displaying "Block all public access: On".
- **Security Benefit:** Hard protection against bucket misconfiguration and unauthorized external downloads.
- **Limitations:** Does not restrict authorized IAM principals within the same account unless bucket policies explicitly restrict them.

6. S3 Default Server-Side Encryption (SSE-S3)

- **Security Objective:** Protect data-at-rest against physical storage media theft or unauthorized access.
- **Configuration:** Enabled S3-managed keys (SSE-S3 / AES-256) default encryption on the target bucket.
- **Evidence:** S3 Bucket Properties tab verifying Encryption configuration set to Amazon S3-managed keys.
- **Security Benefit:** All objects are encrypted automatically upon upload without application code overhead.
- **Limitations:** SSE-S3 does not support customer-managed key rotation or granular KMS key policy audits (which require SSE-KMS).

7. Application Configuration & Environment Variable Protection

- **Security Objective:** Keep runtime options dynamically configurable without modifying codebase.
- **Configuration:** Used .env files (excluded from Git via .gitignore) and system environment variables for port and bucket naming.
- **Evidence:** Repository .gitignore file contains .env entry; zero environment secrets present in public Git commits.

- **Security Benefit:** Prevents operational secrets and environment specifics from leaking into public Git repositories.
- **Limitations:** Environment variables are accessible in cleartext to local processes on the host.

8. Administrative Bearer-Token Authorization

- **Security Objective:** Protect administrative endpoints (e.g., tournament report upload) from unauthorized access.
- **Configuration:** Express middleware verifies Authorization: Bearer <token> header against ADMIN_TOKEN.
- **Evidence:** Recorded HTTP 403 response on missing/invalid token during live test checks.
- **Security Benefit:** Blocks unauthenticated clients from triggering privileged application routes.
- **Limitations:** Prototype mechanism using a shared secret token; lacks multi-user identity management, JWT expiration, or OAuth2 workflows.

9. Input Validation & Error Handling

- **Security Objective:** Sanitize user inputs for court bookings and prevent application crashes or detailed stack trace leaks.
- **Configuration:** Server-side field presence, type, and format validation; generic global error handler returning clean JSON responses.
- **Evidence:** Unit tests verifying HTTP 400 response on malformed booking payloads.
- **Security Benefit:** Protects against malformed data submission and diagnostic data leakage to external clients.
- **Limitations:** Validation logic is custom-coded rather than using a strict schema-validation library like Joi or Zod.

10. HTTP Security Headers and Content Security Policy (CSP)

- **Security Objective:** Protect client browsers against XSS, clickjacking, MIME sniffing, and drive-by downloads.

- **Configuration:** Integrated helmet() middleware in Express.js.
- **Evidence:** Response headers inspecting X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN, and CSP headers.
- **Security Benefit:** Enforces browser-level security policies automatically on all HTTP responses.
- **Limitations:** Default Helmet CSP rules may require custom adjustments when integrating external CDN scripts or fonts.

11. CloudTrail Event History & Operational Audit Evidence

- **Security Objective:** Ensure management actions taken on AWS resources are recorded for post-incident investigation.
- **Configuration:** Leveraged default AWS CloudTrail Event History (covers management events across the account for 90 days).
- **Evidence:** CloudTrail Event History log entries showing IAM Role Assume and S3 API calls.
- **Security Benefit:** Provides verifiable trail of IAM role assume events and S3 bucket modifications without additional infrastructure costs.
- **Limitations:** Does not log S3 data-level events (s3:PutObject calls) by default unless a paid custom trail with S3 Data Events is configured.

12. Secure Report Uploads to S3

- **Security Objective:** Ensure uploaded files are validated before transmission to object storage.
- **Configuration:** Endpoint buffers file content, checks file extensions, and uses AWS SDK PutObjectCommand with proper MIME types.
- **Evidence:** Target S3 bucket object listing verifying uploaded report artifacts with proper metadata.
- **Security Benefit:** Decouples file storage from application web server disk, preventing local file execution exploits.
- **Limitations:** File scanning for malware/viruses is not currently performed prior to S3 upload.

CHAPTER 6: APPLICATION IMPLEMENTATION

6.1 Major Application Components

- app.js: Main Express.js application initialization, Helmet middleware registration, and route definitions.
- routes/bookings.js: Logic for querying court availability, creating new bookings, and handling validation.
- routes/admin.js: Administrative route handlers, bearer token verification middleware, and S3 upload handler.
- services/s3Service.js: Encapsulated AWS SDK v3 S3 client initialized using default IAM credentials.

6.2 Key Security Workflows

IAM Role Integration (No Permanent Keys)

The application initializes the S3 client without passing explicit credentials:// services/s3Service.js

```
const { S3Client, PutObjectCommand } = require('@aws-sdk/client-s3');

// S3 client automatically picks up credentials from EC2 IMDS IAM Role
const s3Client = new S3Client({ region: process.env.AWS_REGION || 'us-east-1' });

async function uploadReportToS3(bucketName, key, fileBuffer, contentType) {
  const command = new PutObjectCommand({
    Bucket: bucketName,
```

```
    Key: key,
    Body: fileBuffer,
    ContentType: contentType,
  });
  return await s3Client.send(command);
}
```

Administrative Bearer Token Middleware

```
// middleware/auth.js
function requireAdminAuth(req, res, next) {
  const authHeader = req.headers['authorization'];
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(403).json({ error: 'Forbidden: Missing or invalid token format' });
  }
  const token = authHeader.split(' ')[1];
  if (token !== process.env.ADMIN_TOKEN) {
    return res.status(403).json({ error: 'Forbidden: Invalid administrative token' });
```

```
  }

  next();

}
```

(Note: Secrets and actual tokens are stored strictly in environment variables and are never committed to the GitHub repository).

CHAPTER 7: TESTING AND VALIDATION

7.1 Test Strategy

Testing was executed in two distinct tiers:

1. Automated Application Test Suite: Executed locally and in CI environment using the Node.js native test runner (node --test).
2. Live AWS Infrastructure Verification: Manual and scripted HTTP checks executed against the live EC2 instance and S3 environment.

7.2 Automated Test Results

The automated application test suite executed **23 test cases with 0 failures (100% pass rate)**.

Table 7.1: Automated Application Test Suite Results

Test ID	Test Description	Expected Result
TEST-01	GET / Dashboard endpoint response	HTTP 200, returns HTM…
TEST-02	GET /api/bookings list availability	HTTP 200, array of booki…

Test ID	Test Description	Expected Result
TEST-03	POST /api/bookings valid input payload	HTTP 201, booking confi…
TEST-04	POST /api/bookings missing required fields	HTTP 400 Bad Request
TEST-05	POST /api/bookings invalid date format	HTTP 400 Bad Request
TEST-06	POST /api/bookings invalid court ID	HTTP 400 Bad Request
TEST-07	POST /api/bookings duplicate time slot booking	HTTP 409 Conflict
TEST-08	POST /api/admin/upload without Auth header	HTTP 403 Forbidden
TEST-09	POST /api/admin/upload with incorrect Bearer token	HTTP 403 Forbidden
TEST-10	POST /api/admin/upload with valid Bearer token	HTTP 200 Success
TEST-11	POST /api/admin/upload missing file attachment	HTTP 400 Bad Request
TEST-12	GET /unknown-route handling	HTTP 404 Not Found
TEST-13	Verify Helmet security header presence	Headers present in response

Test ID	Test Description	Expected Result
TEST-14	Verify Content-Security-Policy header	CSP header present
TEST-15	Verify X-Frame-Options SAMEORIGIN header	Header present
TEST-16	Verify X-Content-Type-Options nosniff	Header present
TEST-17	Sanitize HTML tags in booking user fields	HTML tags stripped/esca…
TEST-18	Handle large string payload injection	HTTP 400 or payload cap…
TEST-19	Verify global error handler middleware	Clean JSON response, no …
TEST-20	Verify JSON body parser middleware	Correct object parsing
TEST-21	Test S3 upload helper mock unit test	S3 client send called
TEST-22	Test environment variable loader	Port and Env set
TEST-23	Graceful shutdown signal handling	Server closes cleanly

7.3 Live AWS Infrastructure Checks

Table 7.2: Live AWS Infrastructure Checks Summary

Check ID	Target Component	Verification Description
LIVE-01	EC2 Systemd Service	Check systemctl status pickleball-app on EC2
LIVE-02	Public Web Endpoint	HTTP GET request to EC2 public IP
LIVE-03	Authorization Gate	HTTP POST to /api/admin/upload without token
LIVE-04	S3 Bucket Storage	Query S3 bucket list via AWS CLI
LIVE-05	HTTPS / TLS Binding	SSL Certificate check on Port 443
LIVE-06	CloudWatch Metric Alarms	Check custom metric filter alarms

CHAPTER 8: CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The **Pickleball Court Management System** successfully demonstrates the implementation of fundamental cloud security controls on AWS within an academic mini-project framework. By replacing permanent access keys with EC2 IAM Roles, enabling S3 Block Public Access with default SSE-S3 encryption, hardening EC2 network security groups, enforcing administrative bearer token checks, and applying Express.js security headers, the system eliminates the primary vulnerabilities associated with cloud web deployments. The project passed 23 out of 23 automated security and functional unit tests, with key operational checks verified live on AWS.

8.2 Future Scope & Recommendations

To transition this prototype toward production readiness, the following enhancements are recommended:

1. **SSL/TLS Encryption:** Bind a custom domain and configure HTTPS via AWS Certificate Manager (ACM) and an Application Load Balancer (ALB) or CloudFront.
2. **Robust Identity Provider:** Replace the prototype shared bearer token with OAuth2 / OpenID Connect (e.g., AWS Cognito or Auth0) for multi-user authentication.
3. **Advanced Audit & Monitoring:** Upgrade CloudTrail to include S3 Data Events and configure custom CloudWatch Log Metric Filters with SNS email alerts for unauthorized access attempts.
4. **Automated CI/CD Security Scanning:** Integrate SAST (Static Application Security Testing) tools such as Semgrep or SonarQube into GitHub Actions workflows.

APPENDIX A: STEP-BY-STEP LIVE DEMONSTRATION PLAN

This section outlines the operational script for conducting the live evaluator demonstration during project evaluation.

7. **EC2 Instance Health & Isolation Check:** SSH into the host using restricted keys. Execute systemctl status pickleball-app to verify active service status on EC2.
8. **IAM Role Credentials Verification:** Query the instance metadata endpoint via curl http://169.254.169.254/latest/meta-data/iam/security-credentials/PickleballEC2S3Role to confirm ephemeral security credentials.
9. **S3 Security Controls Validation:** Open AWS S3 Console; demonstrate that 'Block Public Access' is enabled and SSE-S3 encryption is set as default.
10. **Application Endpoint & Security Headers Check:** Send HTTP GET request to web dashboard. Inspect headers via cURL (curl -I http://<EC2_IP>/) to confirm Helmet headers (CSP, HSTS, X-Content-Type-Options).
11. **Authorization Failure Demonstration:** Attempt to post a report upload to /api/admin/upload without an Authorization header; observe HTTP 403 Forbidden response.

12. **Successful Administrative Upload & Storage Verification:** Submit valid upload request with correct Bearer token. Verify object creation in S3 bucket via AWS CLI (aws s3 ls s3://pickleball-reports-bucket).
13. **Automated Test Suite Execution:** Run npm test on host to show 23 passing automated unit/security tests.
14. **Audit Trail Demonstration:** Open AWS CloudTrail Event History; filter events by IAM Role assume and S3 management calls to demonstrate auditability.

REFERENCES

1. Amazon Web Services, "AWS Cloud Security Best Practices," AWS Whitepapers, 2024.
2. OWASP Foundation, "OWASP Top Ten Web Application Security Risks," 2021.
3. Express.js Security Best Practices, "Production Best Practices: Security," Expressjs.com.
4. AWS IAM Documentation, "IAM Roles for Amazon EC2," Amazon Web Services Documentation.
5. GitHub Repository: https://github.com/Harmann60/pickleball-cloud-security

```
harman@harman-HP-EliteBook-840-G6:~$ cd ~/Documents/pickle
harman@harman-HP-EliteBook-840-G6:~/Documents/pickleball-c

> pickleball-cloud-security@0.1.0 test
> node --test tests/*.test.js

✔ GET /api/dashboard returns fictional data (44.583651ms)
✔ POST /api/bookings creates a valid booking (14.868559ms)
✔ POST /api/bookings rejects invalid input with 400 (4.065
✔ POST /api/bookings rejects double-booking of same slot (
✔ admin endpoint denies missing token (5.987405ms)
✔ admin endpoint denies wrong token (6.901095ms)
✔ admin endpoint allows correct token (2.742169ms)
✔ admin report upload stores locally in test mode (7.41357
✔ admin report upload denies unauthenticated request (3.71
✔ unknown route returns JSON 404 (2.720583ms)
✔ passes through with no CORS headers when ALLOWED_ORIGIN
✔ sets CORS headers for the exact allowed origin (0.255214
✔ answers a preflight request from the allowed origin with
✔ rejects a preflight request from a disallowed origin (0.
✔ accepts a well-formed booking (1.466518ms)
✔ rejects missing fields (0.41018ms)
✔ rejects script injection in playerName (0.175636ms)
✔ rejects malformed email (0.144879ms)
✔ rejects unknown courtId (0.218868ms)
✔ rejects past dates (0.188124ms)
✔ rejects invalid date format (0.203319ms)
✔ rejects invalid time format (0.138837ms)
✔ rejects unexpected durations (0.366733ms)
✔ rejects non-numeric duration (0.375413ms)
✔ report accepts valid content (0.321042ms)
✔ report rejects private key material (0.135727ms)
✔ report rejects short titles (0.123984ms)
ℹ tests 27
ℹ suites 0
ℹ pass 27
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 256.05396
```

Permissions defined in this policy Info

Permissions defined in this policy document specify which actions are allowed or denied. To define permiss



```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "UploadTournamentReports",
            "Effect": "Allow",
            "Action": [
                "s3:PutObject"
            ],
            "Resource": "arn:aws:s3:::pickleball-cloud-security-harman-4339857
        }
    ]
```

Default encryption

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type Info

Server-side encryption with Amazon S3 managed keys (SSE-S3)

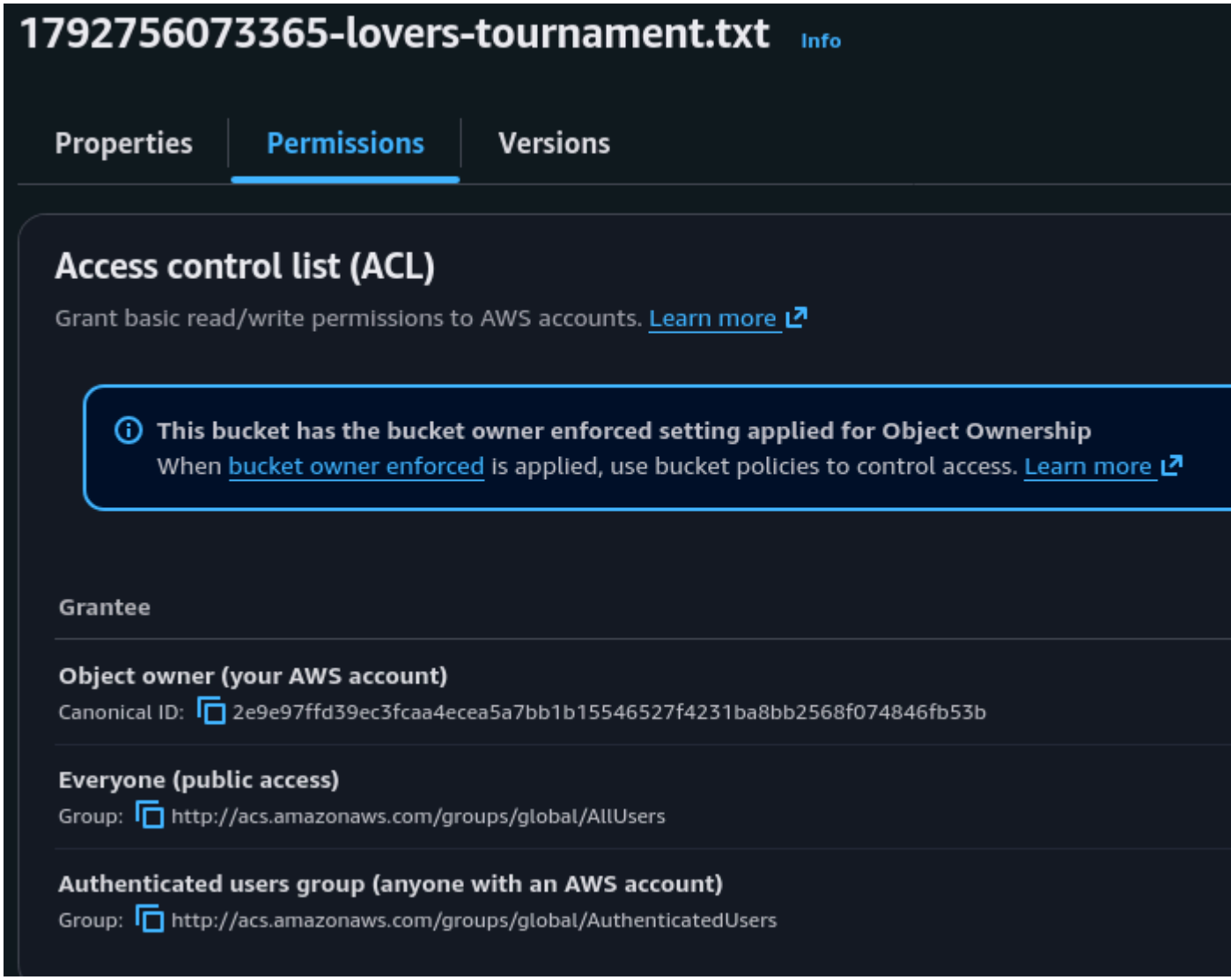
Bucket Key

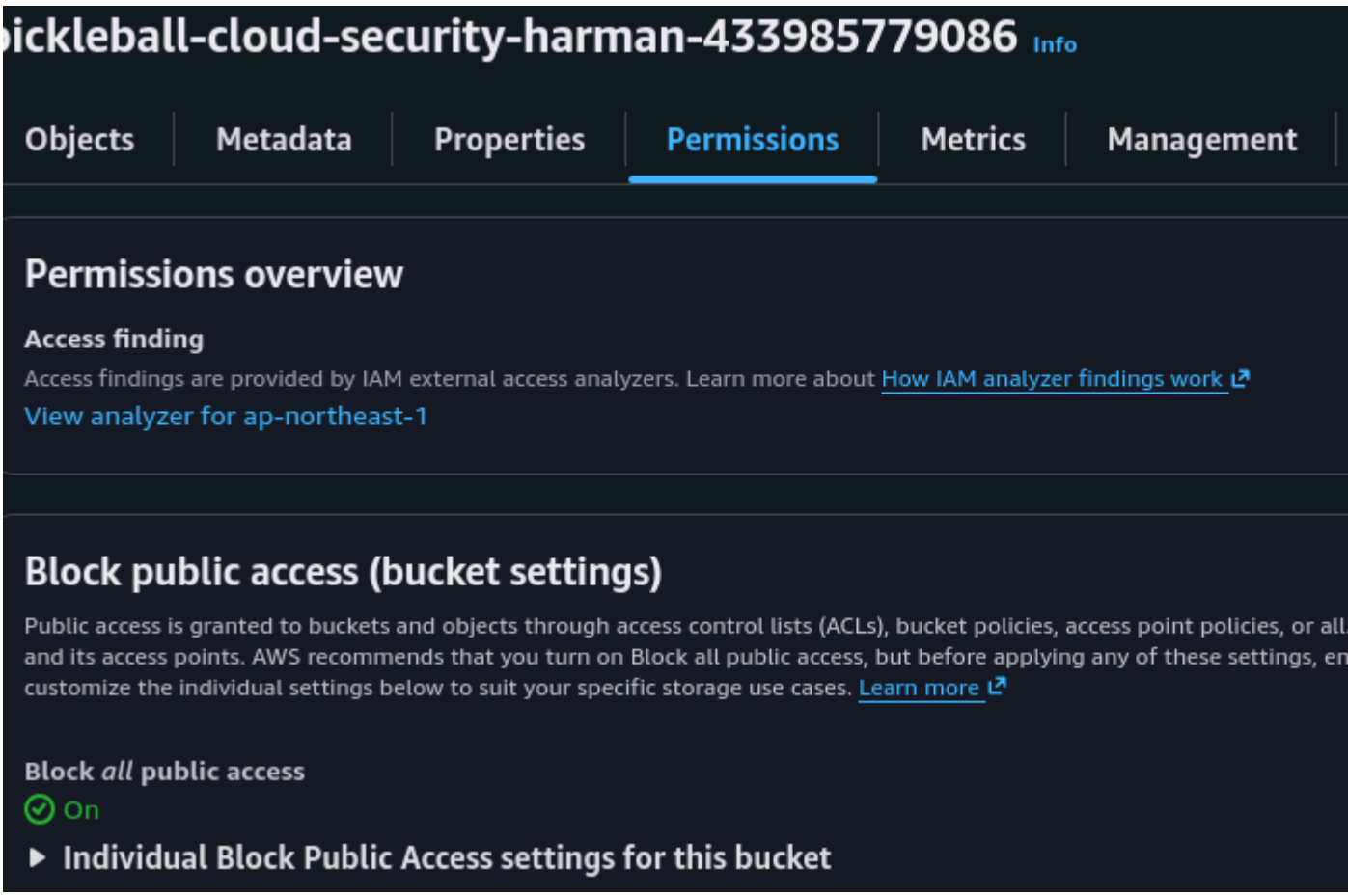
When KMS encryption is used to encrypt new objects in this bucket, the bucket key reduces encryption costs by lowering ca

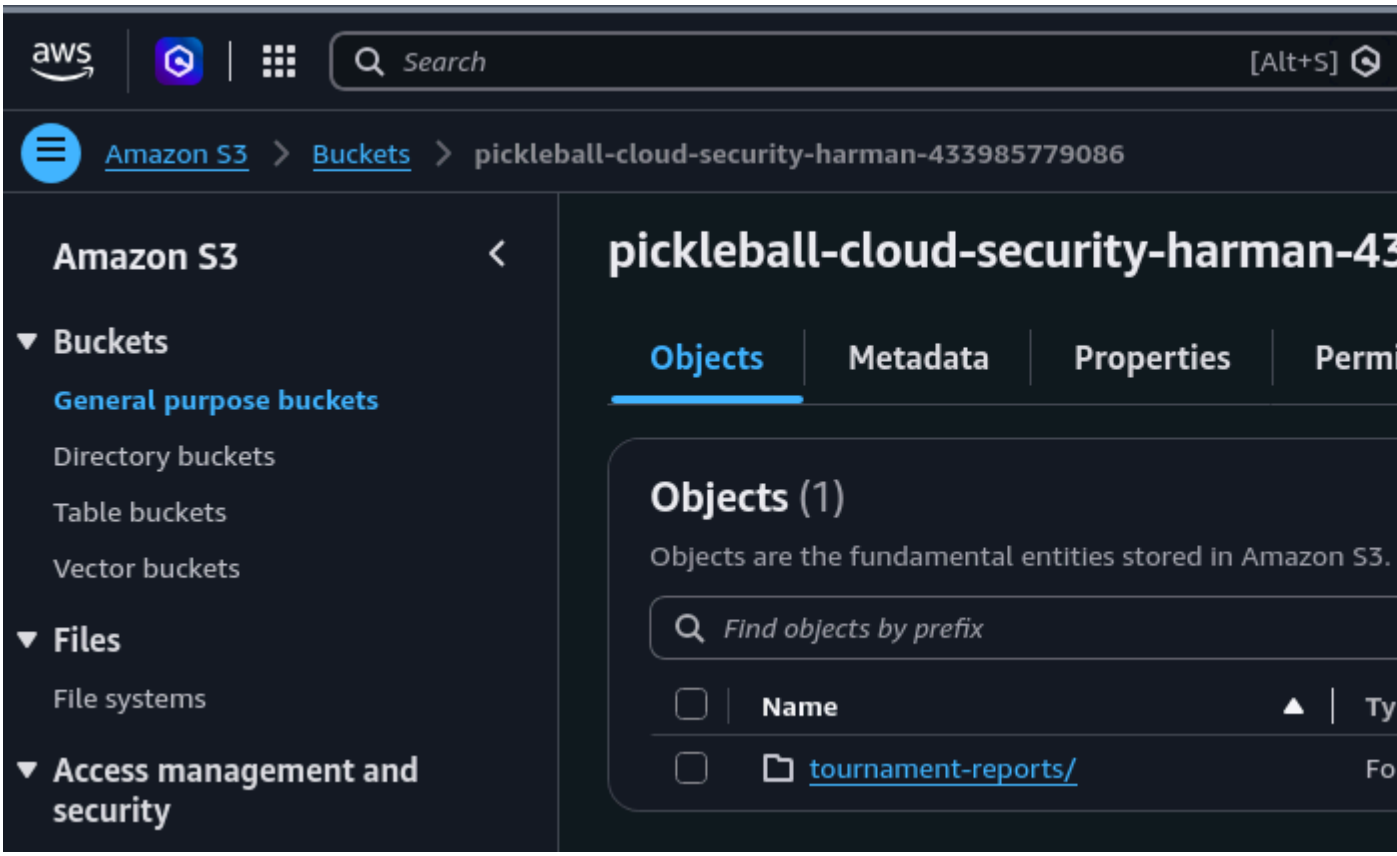
Enabled

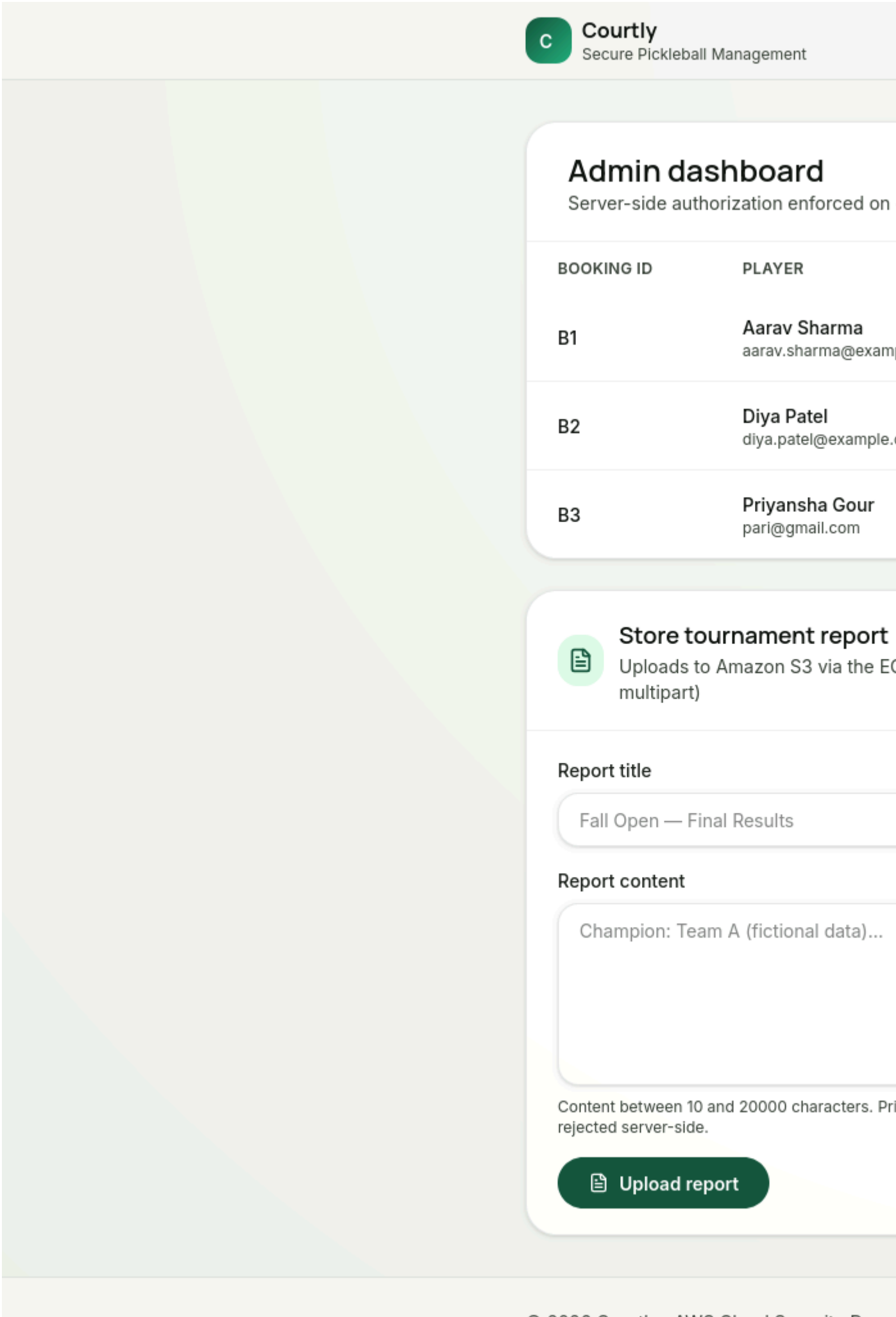
Blocked encryption types Info

Server-Side Encryption with Customer-Provided Keys (SSE-C)

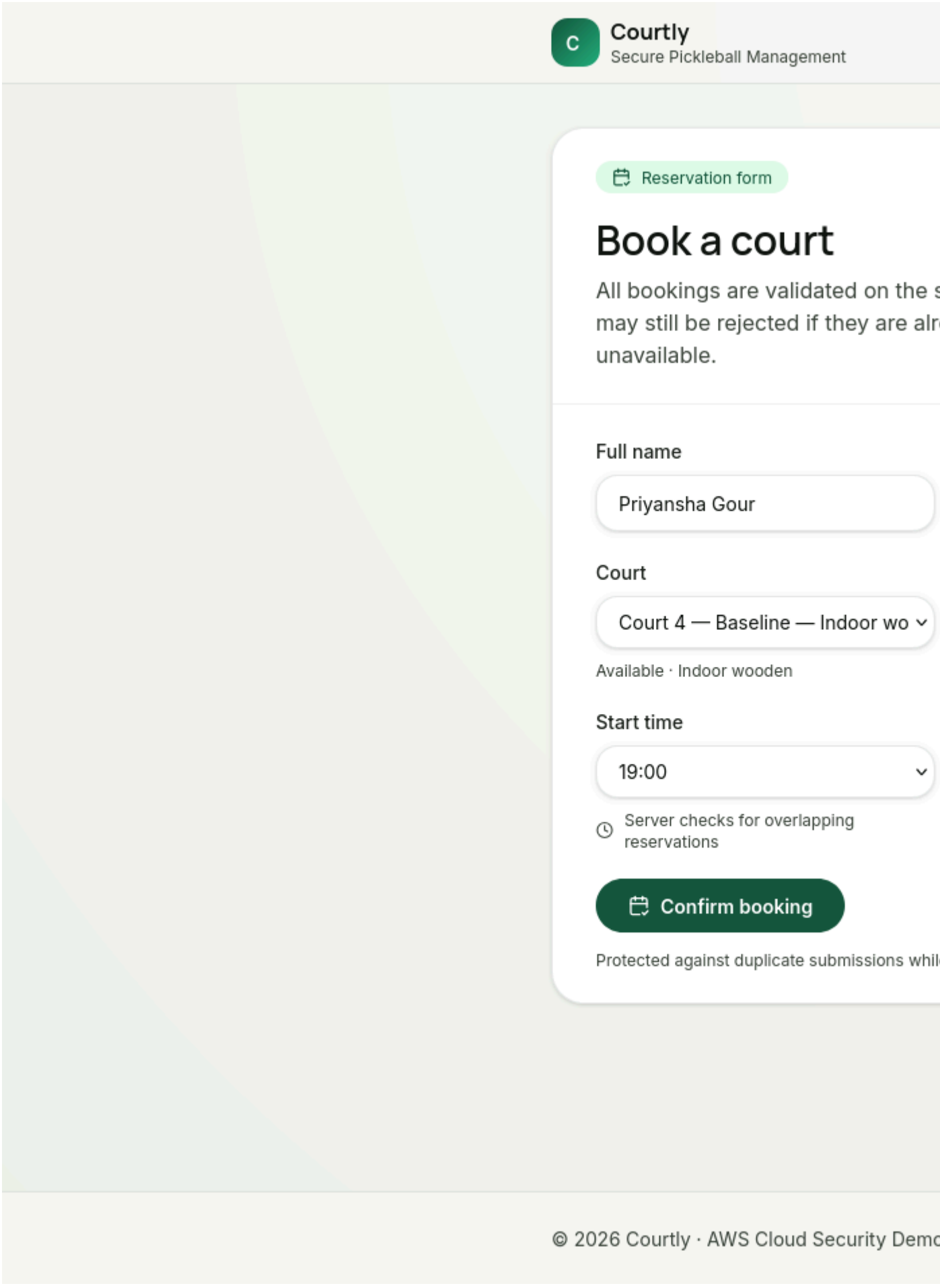

1792756073365-lovers-tournament.txt
Info
Properties
Permissions
Versions
Access control list (ACL)
Grant basic read/write permissions to AWS accounts. Learn more
This bucket has the bucket owner enforced setting applied for Object Ownership
When bucket owner enforced is applied, use bucket policies to control access. Learn more
Grantee
Object owner (your AWS account)
Canonical ID: 2e9e97ffd39ec3fcaa4ecea5a7bb1b15546527f4231ba8bb2568f074846fb53b
Everyone (public access)
Group: http://acs.amazonaws.com/groups/global/AllUsers
Authenticated users group (anyone with an AWS account)
Group: http://acs.amazonaws.com/groups/global/AuthenticatedUsers

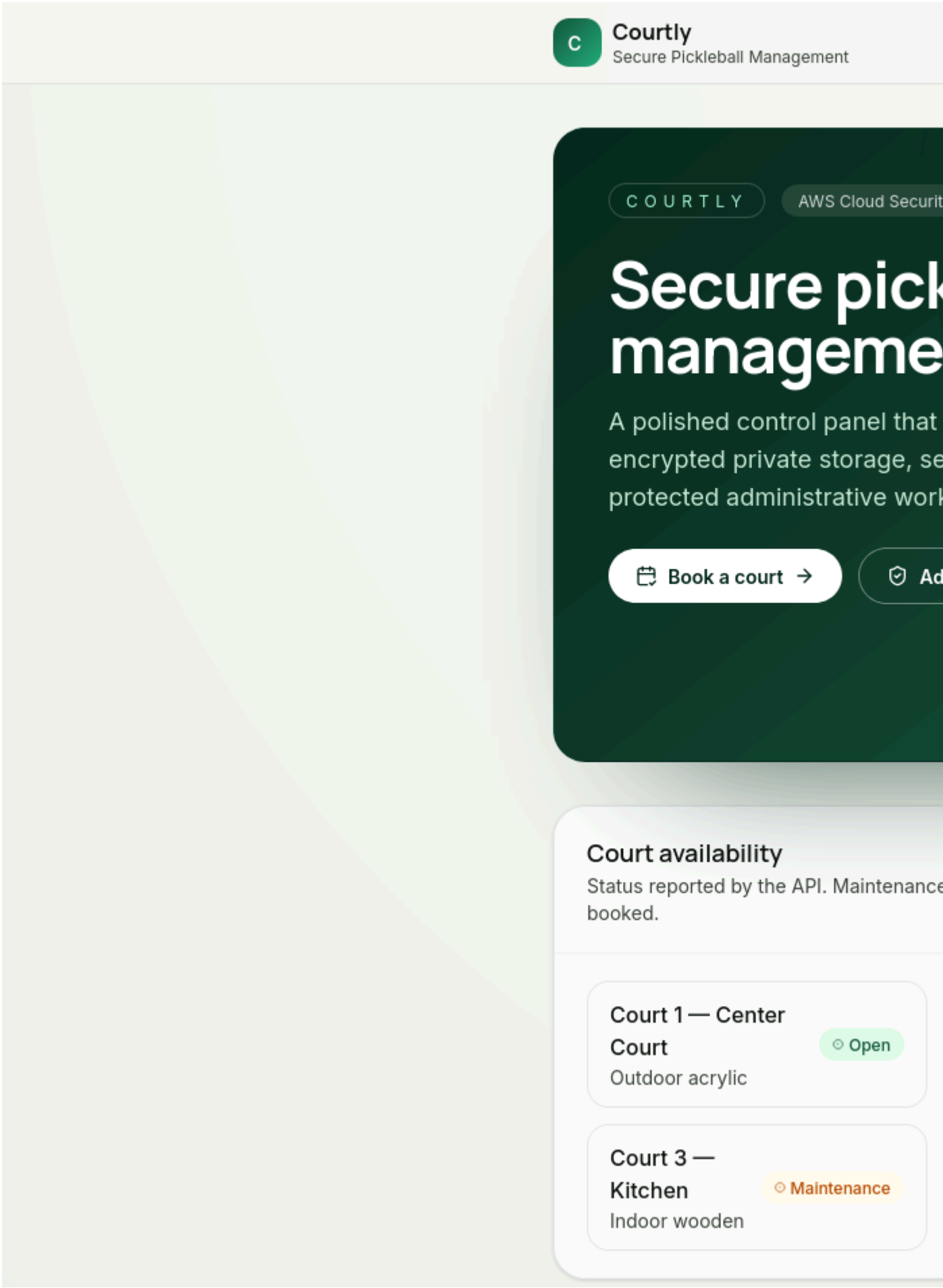

ickleball-cloud-security-harman-433985779086 Info
Objects
Metadata
Properties
Permissions
Metrics
Management
Permissions overview
Access finding
Access findings are provided by IAM external access analyzers. Learn more about How IAM analyzer findings work
View analyzer for ap-northeast-1
Block public access (bucket settings)
Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all
and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, en
customize the individual settings below to suit your specific storage use cases. Learn more
Block all public access
On
Individual Block Public Access settings for this bucket


aws
Search
[Alt+S]
Amazon S3
Buckets
pickleball-cloud-security-harman-433985779086
Amazon S3
Buckets
General purpose buckets
Directory buckets
Table buckets
Vector buckets
Files
File systems
Access management and security
pickleball-cloud-security-harman-43
Objects
Metadata
Properties
Perm
Objects (1)
Objects are the fundamental entities stored in Amazon S3.
Find objects by prefix
Name
Ty
tournament-reports/
Fo


C
Courtly
Secure Pickleball Management
Admin dashboard
Server-side authorization enforced on
BOOKING ID
PLAYER
B1
Aarav Sharma
aarav.sharma@exam
B2
Diya Patel
diya.patel@example.
B3
Priyansha Gour
pari@gmail.com
Store tournament report
Uploads to Amazon S3 via the EC
multipart)
Report title
Fall Open — Final Results
Report content
Champion: Team A (fictional data)...
Content between 10 and 20000 characters. Pri
rejected server-side.
Upload report

© 2026 Courtly · AWS Cloud Security Demo


C
Courtly
Secure Pickleball Management
Reservation form
Book a court
All bookings are validated on the s
may still be rejected if they are alr
unavailable.
Full name
Priyansha Gour
Court
Court 4 — Baseline — Indoor wo
Available · Indoor wooden
Start time
19:00
Server checks for overlapping reservations
Confirm booking
Protected against duplicate submissions whil
© 2026 Courtly · AWS Cloud Security Demo


C
Courtly
Secure Pickleball Management
COURTLY
AWS Cloud Securit
Secure pick
manageme
A polished control panel that
encrypted private storage, se
protected administrative work
Book a court
Ad
Court availability
Status reported by the API. Maintenance
booked.
Court 1 — Center Court
Open
Outdoor acrylic
Court 3 — Kitchen
Maintenance
Indoor wooden